影像處理實習

Week 10. 影像復原與重建

助教 陳立堯

Part 0. 將之前的code類別化

共同父類別

```
class Noise:
  def apply(self, img):
    raise NotImplementedError("子類別須實作apply")
```

類別1 雜訊

```
#高斯雜訊
class GausissionNoise(Noise):
  def __init__(self, mean=0, sigma=20):
    self.mean = mean
    self.sigma = sigma

  def apply(self, img):
    noise = np.random.normal(self.mean, self.sigma,
img.shape)
    noisy = img + noise
    return np.clip(noisy, 0, 255).astype(np.uint8)
```

類別2 雜訊

```
# 椒鹽雜訊
class SaltPepperNoise(Noise):
  def __init__(self, prob=0.05):
    self.prob = prob

  def apply(self, img):
    noisy = img.copy()
    mask = np.random.rand(*img.shape)
    noisy[mask < self.prob] = 0
    noisy[mask > 1 - self.prob] = 255
    return noisy
```

Part 0. 將之前的code類別化

```
# 測試雜訊
img = cv2.imread("/content/Cameraman.tif", cv2.IMREAD_GRAYSCALE)
gaussian = GausissionNoise(0, 20) #實體化 (也可說建立物件)
gaussian_img = gaussian.apply(img) #使用類別方法 (生成雜訊mask)

saltpepper = SaltPepperNoise(0.05)
saltpepper_img = saltpepper.apply(img)

poisson = PoissonNoise()
poisson_img = poisson.apply(img)
```

Part 0. 影像退化程序

一張原本完美的影像 ，經過一個退化(函數)的作用，

再加上加性最後變成了我們看到的爛影像 $g(x,y)$。

我們的目標是在已知H和加性雜訊的特性，從$g(x,y)$逆推回$\hat{f}(x,y)$

$$g(x,y) = H[f(x,y)] + \eta(x,y)$$

退化函數　　加性雜訊

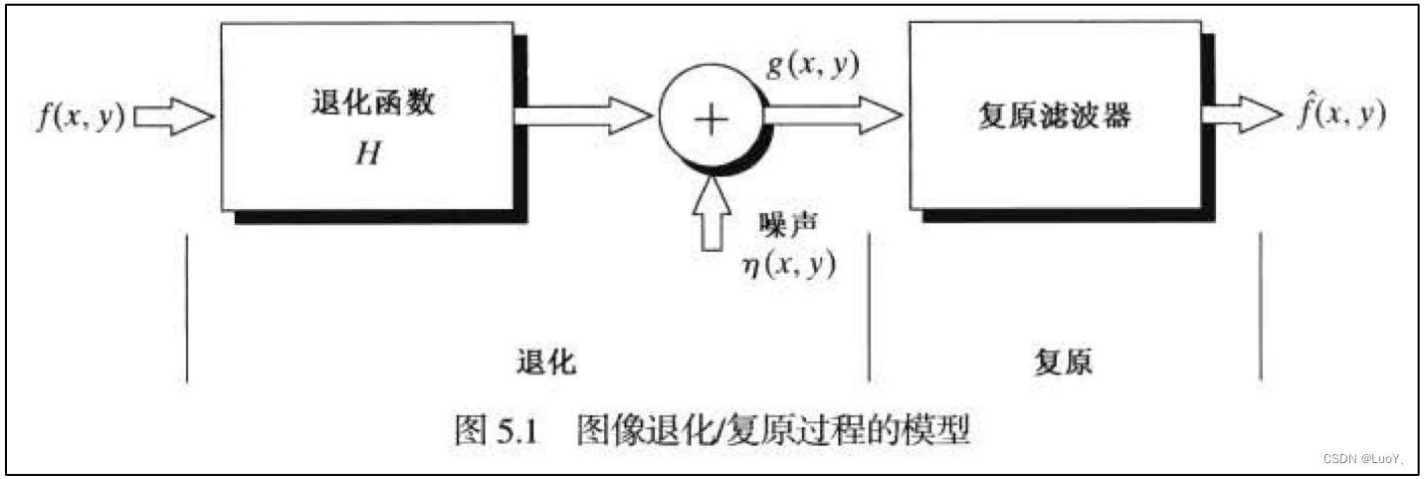

图 5.1　图像退化/复原过程的模型

$$G(u,v) = H(u,v)F(u,v) + N(u,v)$$

影像退化

Part 1. 只有雜訊時的復原

回顧：如果影像沒有模糊（即 H=1），只有雜訊，我們可以使用空間濾波器來復原。

```
#高斯雜訊
class  GausissionNoise(Noise):
    def  __init__(self,  mean=0,  sigma=20):
        self.mean  =  mean
        self.sigma  =  sigma
```

```
#椒鹽雜訊
class  SaltPepperNoise(Noise):
    def  __init__(self,  prob=0.05):
        self.prob  =  prob
    def  apply(self,  img):
        noisy  =  img.copy()
```

```
#泊松雜訊
class  PoissonNoise(Noise):
        def  __init__(self):
                pass

        def  apply(self,  img):
                #  正規化到  [0,1]
                img_norm  =  img  /  255.0
```

```
#週期性雜訊
class  PeriodicNoise(Noise):
        def  __init__(self,  amplitude=30,  freq_x=0.05,  freq_y=0.05):
                self.amplitude  =  amplitude
                self.freq_x  =  freq_x
                self.freq_y  =  freq_y
```

Part 2. 估測退化函數 - **運動模糊**

```python
def get_motion_blur_H(shape, length=20, angle=0):
    M, N = shape
    H = np.zeros((M, N), dtype=np.float32) #建立模糊核
    center = (N // 2, M // 2) #計算中心座標
    angle_rad = np.deg2rad(angle)

    dx = int((length / 2) * np.cos(angle_rad))
    dy = int((length / 2) * np.sin(angle_rad))

# (center_x - dx, center_y - dy) → (center_x + dx, center_y + dy)
# 代表物體的移動方向
    cv2.line(H, (center[0] - dx, center[1] - dy),
                (center[0] + dx, center[1] + dy), 1, 1)
    # normalize確保影像亮度不變
    H = H / np.sum(H)
    # FFT to get degradation function
    H = np.fft.fftshift(np.fft.fft2(H))
    return H #複數 (表示運動模糊)
```

$$H(u,v) = \frac{T}{\pi(ua+vb)} \sin\left(\pi(ua+vb)\right) e^{-j\pi(ua+vb)}$$

發生於：相機曝光期間，物體或相機**沿直線移動**。

結果：影像在某個方向被拖成「條狀」。

Part 2. 估測退化函數 - 大氣擾動

```python
def get_atmospheric_H(shape, k):
    rows, cols = shape
    center_x, center_y = rows // 2, cols // 2

    # 建立頻率座標網格 (u, v)，u:橫向，v:縱向
    u, v = np.meshgrid(np.arange(cols) - center_y, np.arange(rows) - center_x)

    # 計算距離平方 (u^2 + v^2)
    D2 = u**2 + v**2
    # 套用公式 (注意：為了讓效果明顯，k值需要調整，且距離通常需正規化)
    # 這裡將距離正規化，避免D2指數爆炸
    D2_norm = D2 / (rows * cols) * 10000
    H = np.exp(-k * (D2_norm ** (5/6)))
    return H
```

$$H(u,v) = e^{-k(u^2+v^2)^{5/6}}$$

發生於：光線穿過空氣時，因溫度密度不同造成折射率隨機變化。

影像復原

助教 陳立堯

Part 3. 將影像還原 - Wiener Filter

```python
def wiener_filter(G, H, K=0.001):
    H_conj = np.conj(H)
    F_hat = (H_conj / (np.abs(H)**2 + K)) * G
    return F_hat
```

一般逆濾波

$$\hat{F} = \frac{G}{H}$$

Wiener

$$\hat{F}(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K} G(u,v)$$

當H很小 → 噪聲過度放大 → 有K值保底

當H很大 → 無太大影響 → 1/H 直接還原

本日作業

- 請練習兩個估測退化函數、Wiener Filter
- 將練習程式碼截圖後繳交